

Análisis de Sistemas

Materia:
Análisis y Metodología de
Sistemas

Docente contenidista: QUIRÓZ, Gamaliel

Revisión: Coordinación

Contenido

¿Qué es un diccionario de datos?	04
¿Para qué sirve?	04
¿Qué se documenta en un diccionario de datos?	04
Recomendaciones para mejorar la legibilidad de un diccionario de datos	06
Diccionario de Datos:	07
Interfaces de Usuario (UI) y Experiencia de Usuario (UX)	07
Interfaces de Usuario	08
Bibliografía	17

Clase 4



¡Te damos la bienvenida a la materia
Análisis y Metodología de Sistemas!

En la clase anterior vimos muchos conceptos sobre la experiencia de usuario y sobre el final vimos el manual de usuario, teniendo eso en cuenta comenzaremos ya viendo lo que es el diccionario de datos y avanzamos. ¡¡Vamos!!.

En esta clase vamos a ver los siguientes temas:

- El rol del diccionario de datos en el análisis de sistemas.
- Principios de diseño de interfaces y experiencia de usuario.
- Buenas prácticas para formularios, consistencia visual y feedback.
- Uso de listas y funciones en Python.

¿Qué es un diccionario de datos?

Un diccionario de datos es una herramienta fundamental en el análisis y diseño de sistemas de información. Consiste en una recopilación organizada y sistemática de todos los datos que forman parte de un sistema, junto con sus respectivas definiciones, estructuras y relaciones.

¿Para qué sirve?

- ❖ Garantiza que **usuarios y analistas** compartan un mismo entendimiento sobre los datos.
- ❖ Aporta **claridad y coherencia** en la definición de:
 - **Entradas y salidas del sistema**
 - **Almacenes de datos**
 - **Procesos intermedios**
- ❖ Evita ambigüedades durante el desarrollo o mantenimiento del sistema.

¿Qué se documenta en un diccionario de datos?

- Flujos de datos: se describe su significado, origen, destino y composición.
- Almacenes de datos: qué tipo de información contienen, cómo se agrupa y qué estructura posee.
- Paquetes de datos: organización interna de los datos que circulan dentro del sistema (registros, campos, claves, etc.).

Notación estandarizada

Para representar los elementos y estructuras del diccionario, se utiliza una notación que facilita la comprensión. A continuación, los símbolos más utilizados:

Símbolo	Significado
=	Está compuesto por
+	Composición obligatoria
()	Opcional
{ }	Repetición (uno o más elementos)
[]	Selección entre alternativas
**	Comentario o aclaración
@	Campo identificador (clave primaria o clave única)

Ejemplo 1: Datos Personales

Vamos a representar cómo se puede definir el campo **"Nombre"** y **"Dirección"** usando la notación:

Nombre = 1er nombre + (2do nombre) + apellido + (2do apellido)

Dirección = calle + altura + (piso + (departamento))

Explicación:

- El segundo nombre y apellido son opcionales.
- Piso y departamento también son opcionales y están anidados, es decir, el departamento solo aparece si hay piso.

Ejemplo 2: Remito de Entrega

Remito = cabecera remito + 1{código + descripción + cantidad}10

Cabecera remito = fecha + nro remito + datos cliente

Explicación:

- El remito se compone de una **cabecera** (que contiene la información principal del documento).
- Luego, entre 1 y 10 **líneas de producto**, cada una con código, descripción y cantidad.

Ejemplo 3: Número de Empleado

Número empleado = 1 + número sucursal + 20

Explicación:

- Esta es una codificación personalizada.
- Se codifica con un dígito inicial, seguido del número de sucursal y un identificador de 20 caracteres.

Recomendaciones para mejorar la legibilidad de un diccionario de datos

- Usar **sangrías y espaciado** adecuado en estructuras complejas.
- Separar visualmente los bloques lógicos.
- Evitar ambigüedades en nombres de campos.
- Agregar **comentarios aclaratorios** donde sea necesario.
- Incluir **ejemplos concretos** cuando los datos sean abstractos o poco familiares.

Ejemplo pasando de DFD a DD:

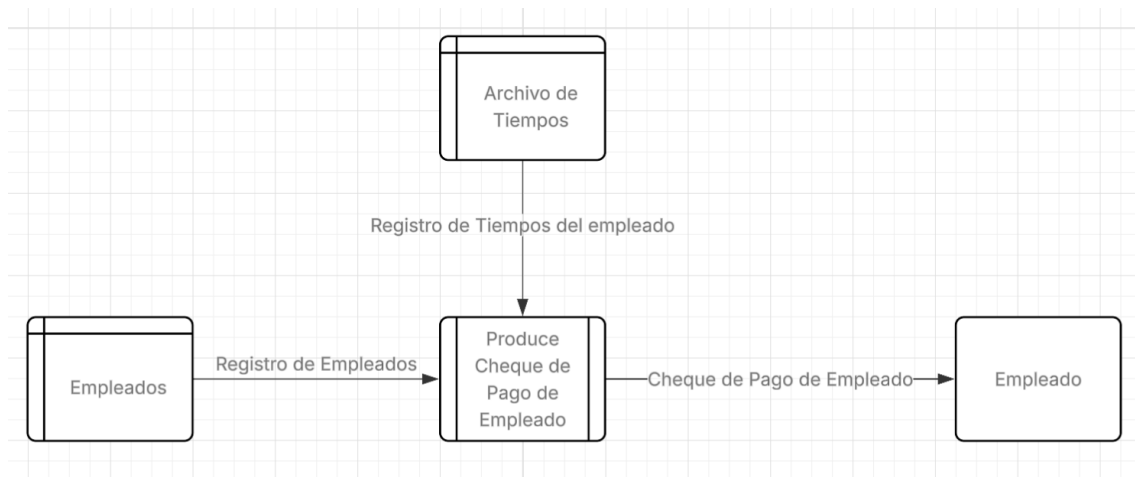


Imagen: Diagrama de flujo de datos y sus entradas del diccionario de datos correspondientes para la producción de un cheque de pago de empleado.

Diccionario de Datos:

Registro de Empleados = Número de Empleado + Información del personal + información del pago + información del pago actual + información anual.

Registro Archivo de Tiempo = Número de empleado + nombre de empleado + Horas trabajadas.

Cheque de pago del empleado = Número de empleados + Nombre de Empleado + Dirección + Cantidades del pago actual + Cifras Anuales.

Interfaces de Usuario (UI) y Experiencia de Usuario (UX)

Ya hemos hablado de esto en clases pasadas, pero vamos a refrescar un poco. La UX se refiere a la experiencia total del usuario al interactuar con un producto, sistema o servicio. Una buena UX no solo mejora la satisfacción del usuario, sino que también aumenta la eficiencia y reduce errores. Usar principios de UX en el diseño de la UI garantiza que los usuarios encuentren la interfaz intuitiva, accesible y agradable.



Interfaces de Usuario

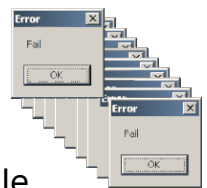
¿Qué son los mockups y por qué son importantes?

Los mockups son representaciones visuales estáticas que simulan cómo lucirá una interfaz antes de ser programada. Son herramientas valiosas en el diseño de sistemas porque:

- Permiten visualizar el diseño anticipadamente.
- Ayudan a detectar errores o inconsistencias de forma temprana.
- Aseguran que las expectativas del usuario y los objetivos del proyecto estén alineados.

Problemas frecuentes en las interfaces

En entornos académicos o proyectos enfocados al backend, suele descuidarse el diseño visual. Sin embargo, la interfaz no debe ser vista como un simple "formulario", sino como un canal de comunicación activo.



Las interfaces deben anticipar, guiar y responder a las acciones del usuario. Algunas consideraciones importantes:

Separar Acciones Complejas en Pasos

Problemas de formularios largos y complejos:

- Formularios extensos pueden resultar abrumadores, lo que lleva a errores, abandonos y frustración del usuario.

Estrategias para dividir formularios en pasos (paginación, secciones):

- Dividir el proceso en pasos o secciones más pequeñas hace que la tarea sea más manejable, mejorando la usabilidad y reduciendo la carga cognitiva del usuario.

Ejemplos de buenos y malos formularios en términos de usabilidad:

Comparar un formulario largo con su versión mejorada, dividida en pasos claros, demostrando cómo mejora la experiencia.

Consistencia en el Diseño

La consistencia en el diseño se refiere al uso uniforme de elementos visuales y patrones de interacción en toda la interfaz, lo que facilita el aprendizaje y el uso del sistema por parte del usuario.

Reduce la curva de aprendizaje, mejora la previsibilidad, y permite a los usuarios concentrarse en el contenido en lugar de en cómo interactuar con la interfaz.

Mantener una tipografía uniforme, colores coherentes, y un diseño consistente de botones y espaciados asegura que los usuarios se sientan cómodos y confiados al navegar por la interfaz.

Comparen dos interfaces, una con elementos consistentes y otra sin consistencia, para demostrar el impacto en la usabilidad.

Feedback al Usuario

El feedback es la respuesta que recibe un usuario tras realizar una acción en la interfaz. Puede ser visual, como un cambio de color, o textual, como un mensaje de confirmación.

Proporcionar feedback inmediato ayuda a los usuarios a entender si sus acciones han sido correctas o si necesitan corregir algo.

El feedback adecuado reduce la incertidumbre y la ansiedad del usuario, mejorando su confianza y eficiencia en el uso de la aplicación.

Brindar una respuesta clara a cada acción mejora enormemente la experiencia.

Ejemplos:

- Un mensaje de confirmación tras enviar un formulario.
- Un campo resaltado cuando hay un error.

No abordaremos mucho en esto ya que es parte del especialista en UX y UI, sin embargo es importante que nosotros como *Analistas de Sistemas* tengamos noción sobre estos conceptos y los apliquemos en el día a día de nuestra programación.



¡¡¡PYTHON!!!!

En esta clase vamos a trabajar sobre las listas y funciones dentro de PYTHON

Arrays en python

En Python no tenemos arrays como en otros lenguajes de programación (Por ejemplo: Java). En su lugar, usamos algo que se llama listas (list). Las listas son súper flexibles: podés guardar lo que quieras dentro, y además se pueden agrandar o achicar según lo necesites.

Creación de una lista:

```
1
2 lista = [1, 2, 3, 4]
3
```

Crear una lista en Python solo tenemos que asignarle un nombre y utilizar los corchetes [].

Acceso de una lista:

```
lista = [1, 2, 3, 4]

lista[0] # Primer elemento
lista[-1] # Último elemento
```

Modificar elementos

Para modificar un elemento tenemos que acceder a él y le asignamos el valor:

```
#Cambiar elementos
lista[2] = 10 # Cambia el valor de la tercera posición
```

Diferencias

En Java los arrays tienen un tamaño fijo. En Python, las listas son dinámicas. Esto significa que pueden cambiar de tamaño (agregar o quitar elementos) sin problemas.

Realmente es una mezcla entre List y arrays. En el material complementario podremos encontrar más documentación sobre esto.

Operaciones Básicas

Para modificar las listas, tenemos las siguientes operaciones que nos provee Python:

- **append(num):** Agrega al final de la lista un elemento.
- **insert(pos, num):** Inserta un elemento en la posición indicada.
- **remove(num):** Elimina el elemento especificado según el valor.
- **lista.pop(num):** Elimina el elemento indicado y lo almacena en una variable.

```
lista.append(6)          # Agrega al final
lista.insert(1, 8)       # Inserta en la posición 1
lista.remove(8)          # Elimina la primera aparición del 8
elemento = lista.pop()   # Elimina y devuelve el último elemento
```

Operaciones Básicas

Buscar elementos en una lista podemos hacerlo de manera directa:

```
if 4 in lista:
    print("4 está en la lista")
```

O utilizando el bucle for para recorrer la lista:

```
for elemento in lista:
    print(elemento)
```

Uso de listas complejas

Una lista en Python realmente tiene una diferencia bastante particular respecto a otros lenguajes. Podemos guardar cualquier tipo de dato sin ningún problema de Sintaxis en absoluto (a diferencia de Java por ejemplo). Veamos:

```
#Listas COMPLEJAS

producto = ["Notebook", 899.99, 10, True]
```

Acá, la lista "producto" guarda el nombre del **producto**, el **precio**, el **stock** disponible, y si está en **oferta (True o False)**. Todos los datos están juntos y son fáciles de manejar.

Otra cosa aún más divertida y que podemos sacarle provecho:

¡Podemos guardar listas de listas! :D

En el siguiente código estamos creando una lista de Productos en la cual estamos asignando dos listas (cada lista es un producto en sí):

```
#Lista de Listas

productos = [
    ["Notebook", 899.99, 10, True],
    ["Mouse", 19.99, 50, False]
]
```

Esto es bastante parecido a usar JSON en Java (sin clave valor).

Recorrer y mostrar toda la información:

Para recorrer una lista es muy sencillo, solo tenemos que usar el bucle for:

```
for producto in productos:
    nombre = producto[0]
    print(f"Producto: {nombre}")
```

Si queremos obtener toda la información de los productos que hemos creado previamente:

```
for producto in productos:
    nombre = producto[0]
    precio = producto[1]
    stock = producto[2]
    oferta = "Sí" if producto[3] else "No"
    print(f"Producto: {nombre}, Precio: {precio}, Stock: {stock}, En oferta: {oferta}")
```

((Muy similar a "foreach"))

Funciones

Para definir una función en Python, usás la palabra clave **def** seguida del nombre de la función y paréntesis ().

Si la función necesita recibir algún dato (parámetros), tendremos que ponerlo entre los paréntesis, esto es igual que en la mayoría de lenguajes de programación. Después, el código que queremos que se ejecute tiene que ir indentado (usando tabulaciones o espacios).

A diferencia de otros lenguajes (Ej: Java), las funciones en Python no necesitan especificar el retorno:

Ejemplos básicos de uso de Funciones:

Creamos una función que imprima por pantalla un mensaje:

```
def saludar():  
    print("Hola, ¿cómo estás?")  
#Vease que así como con las demás estructuras tipo for, if etc tenemos que tabular.
```

Usando funciones utilizando pasaje de parámetros:

Lo que hace "saludar(nombre="sin nombre")" es asignarle un valor por defecto al parámetro recibido, esto en caso de que no le pasemos nada, el nombre pasa a tener valor "sin nombre"

```
def saludar(nombre="sin nombre"):  
    print(f"Hola, {nombre}, ¿cómo estás?")  
  
saludar("Elias")
```

Funciones que reciben varias entradas

También podemos usar algo llamado *args para que una función reciba varios parámetros sin tener que especificar cuántos son de antemano. Esto es muy útil cuando no sabés cuántos valores va a recibir la función.


```
#Múltiples parámetros:
def sumar_todo(*numeros):
    return sum(numeros)

print(sumar_todo(1, 2, 3, 4)) # Devuelve 10
```

((`sum()` es una función interna de Python, conocida como una **built-in function**.

Estas funciones vienen predefinidas en el lenguaje, lo que significa que podés usarlas sin necesidad de importarlas desde ningún módulo o librería.))

Funciones - Uso del return

Veamos lo que pasa cuando no usamos `return`.

En el código la función no tiene una especificación "`return`" para que devuelva un valor, en ese caso, si queremos imprimir su resultado asignando este a una variable, nos devolverá "`none`":

```
def sin_return():
    print("Esto no devuelve nada")

resultado = sin_return()
print(resultado) # Imprime: None
```

Con "`return`"

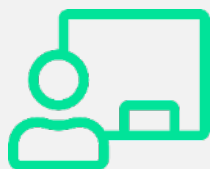
```
def con_return():
    return("Esto devuelve un mensaje")

resultado = con_return()
print(resultado) #Devuelve el mensaje especificado en el return en lugar de imprimir "None"
```



Hemos llegado así al final de esta clase en la que vimos:

- El rol del diccionario de datos en el análisis de sistemas.
- Principios de diseño de interfaces y experiencia de usuario.
- Buenas prácticas para formularios, consistencia visual y feedback.
- Uso de listas y funciones en Python.



Te esperamos en la **clase en vivo** de esta semana.
No olvides realizar el **desafío semanal**.

¡Hasta la próxima clase!

Bibliografía

Pressman, R. S., & Maxim, B. R. (2020). Ingeniería del software: Un enfoque práctico (8.ª ed.). McGraw-Hill.

Yourdon, E. (1989). Modern Structured Analysis. Prentice Hall.

Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., Elmqvist, N., & Diakopoulos, N. (2016). Designing the user interface: Strategies for effective human-computer interaction (6th ed.). Pearson.

Norman, D. A. (2013). The design of everyday things (Revised and expanded ed.). Basic Books.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Sommerville, I. (2011). Ingeniería de software (9ª ed.). Pearson.
https://gc.scalahed.com/recursos/files/r161r/w25469w/ingdelsoftwar elibro9_compressed.pdf

El Libro de Python. (s.f.). Listas en Python.
<https://ellibrodepython.com/listas-en-python>

Harvard Medical School. (s.f.). Data dictionary.
<https://datamanagement-hms-harvard-edu.translate.goog/collect-analyze/documentation-metadata/data-dictionary? x tr sl=en& x tr tl=es& x tr hl=es& x tr pto=tc>

Python Software Foundation. (s.f.). The Python tutorial. Python.org.
<https://docs.python.org/3/tutorial/index.html>

Nielsen Norman Group. (s.f.). Usability heuristics for user interface design. Nielsen Norman Group.
<https://www.nngroup.com/articles/ten-usability-heuristics/>

ISO. (2018). ISO 9241-210: Ergonomics of human-system interaction — Part 210: Human-centred design for interactive systems. International Organization for Standardization.

Object Management Group (OMG). (s.f.). Unified Modeling Language (UML). OMG. <https://www.uml.org>

Kroenke, D. M. (2015). Database processing: Fundamentals, design, and implementation. Pearson Education.